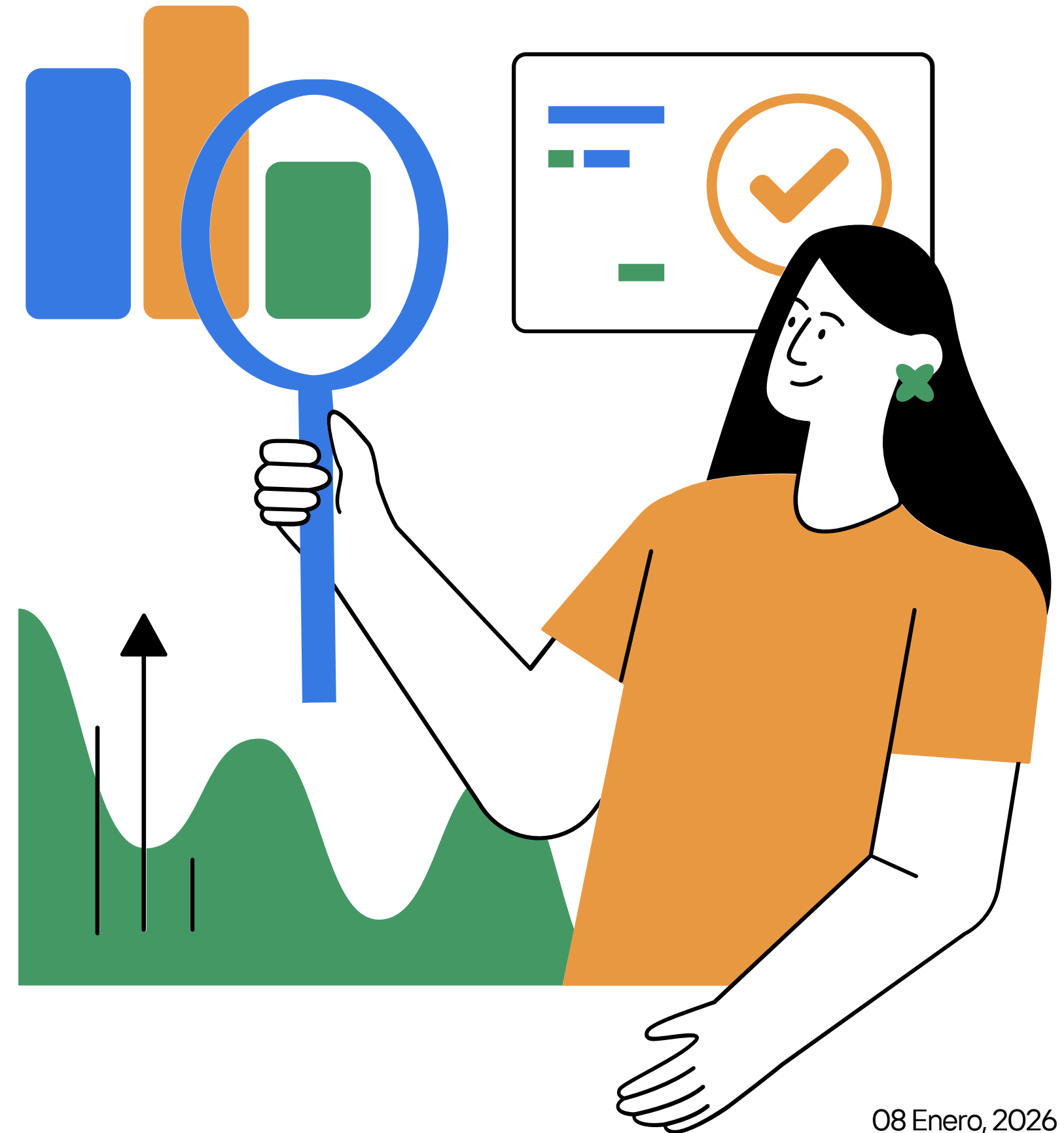
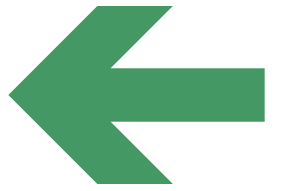


Análisis de sentimientos

- Regular Expressions
- Text Summarization
- Transformer Models





Regular Expressions

Reglas para buscar, limpiar y transformar texto.

¿Por qué es importante aquí?

- Eliminan ruido del texto.
- Normalizan datos para que el modelo entienda mejor.

Qué se limpia:

- ✓ URLs
- ✓ Emojis (según necesidad)
- ✓ Menciones (@user)
- ✓ Repeticiones exageradas ("!!!!!!")

Lookahead:

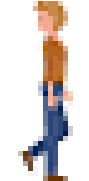
El lookahead permite buscar una palabra solo si está seguida por otro patrón. No incluye ese patrón en el resultado, solo lo usa como condición.

Ejemplo:

Encontrar “bueno” solo si después viene un “!”

Lookahead y Lookbehind

Look-Around in Regex

	Positive	Negative
 Look-Ahead	A (?=B) Find <i>expression A</i> , which has <i>expression B</i> after it.	A (?!B) Find <i>expression A</i> , which does not have <i>expression B</i> after it.
 Look-Behind	(?<=B) A Find <i>expression A</i> , which has <i>expression B</i> before it.	(?<!B) A Find <i>expression A</i> , which does not have <i>expression B</i> before it.

Copyright © 2022 Zeno. All rights reserved.

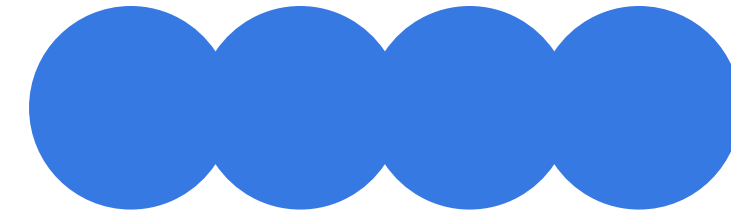
Lookbehind

Párrafo:

El lookbehind permite encontrar una palabra únicamente si antes aparece otro patrón. Este texto previo no se incluye en el resultado, solo actúa como filtro.

Ejemplo:

Encontrar palabras que vienen justo antes de “muy”



Grupos Nombrados

Los grupos nombrados permiten identificar y extraer partes del texto asignándoles un nombre descriptivo. Esto facilita procesar elementos específicos como menciones, URLs o emojis, ya que cada uno queda etiquetado para manipularlo por separado. Son especialmente útiles cuando se necesita limpiar el texto o clasificar diferentes tipos de información dentro de un dataset.



```
import re

texto = "Hola @justin_z, revisa esto por favor."

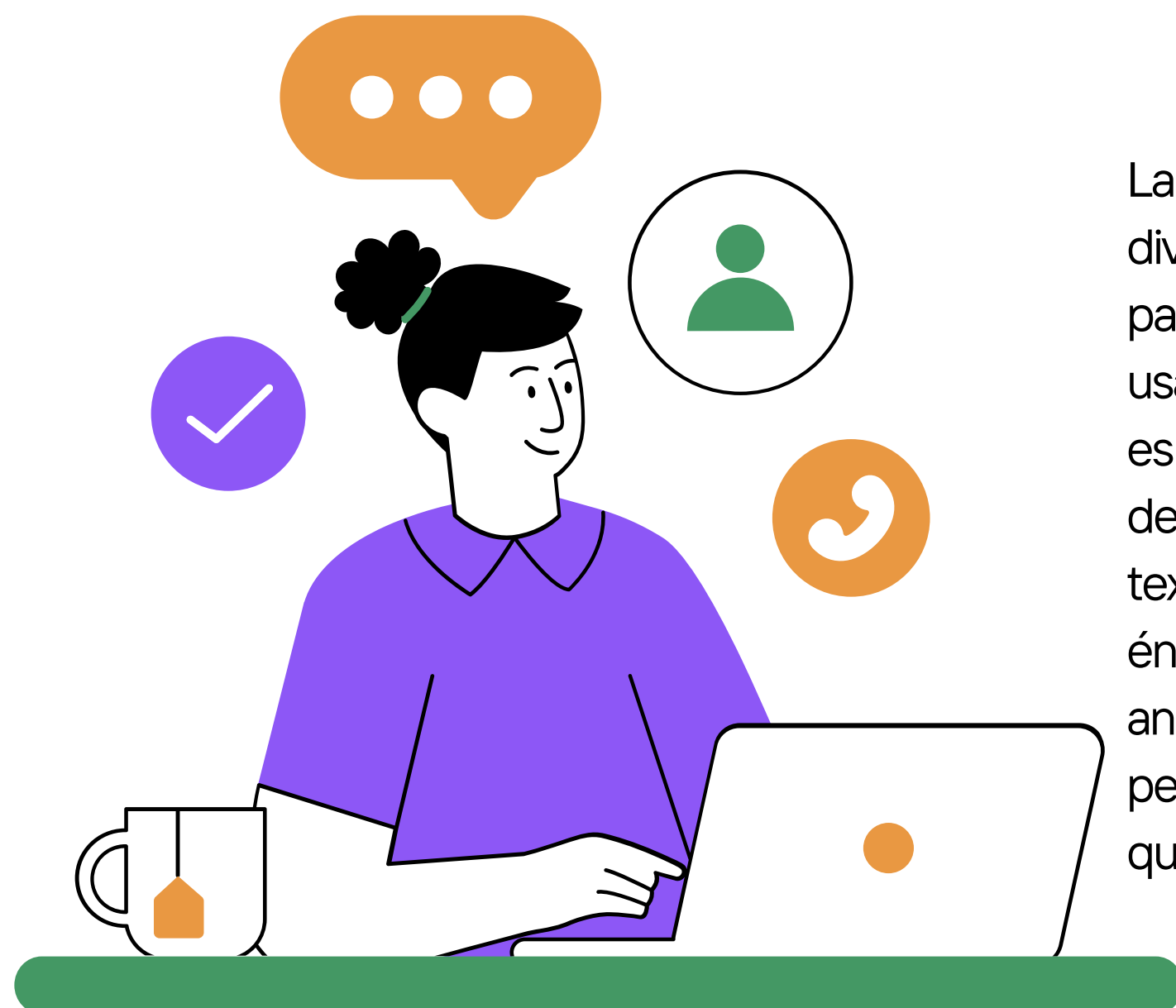
patron = r"(?P<usuario>@[A-Za-z0-9_]+)"
match = re.search(patron, texto)

if match:
    print("Usuario encontrado:", match.group("usuario"))
```

✓ 0.0s

Usuario encontrado: @justin_z

Tokenización con Regex



La tokenización con Regex permite dividir el texto en componentes como palabras, signos de puntuación y emojis usando patrones definidos. Esta técnica es importante porque ayuda a conservar detalles emocionales o semánticos del texto, como los emojis o signos de énfasis, que influyen directamente en el análisis de sentimientos. Además, permite una segmentación más precisa que algunos tokenizadores tradicionales.

```
import re

texto = "Estoy feliz 😊 pero también cansado..."

patron = r"\w+|[\U0001F600-\U0001F64F]"
tokens = re.findall(patron, texto)

print(tokens)
```

✓ 0.0s

```
['Estoy', 'feliz', '😊', 'pero', 'también', 'cansado']
```


Detección Básica de Sarcasmo

Aunque Regex no puede detectar sarcasmo de forma profunda ni interpretar la intención real del autor, sí puede identificar expresiones o patrones que suelen aparecer en mensajes sarcásticos. Esto incluye frases repetidas, ironía escrita o signos de puntuación peculiares. Si bien es una detección limitada, ayuda a señalar textos que podrían tener un tono no literal, lo cual es útil para análisis preliminares.

```
import re

texto = "Oh sí, claro que te creo..."

patron = r"s[íí]\,?\s+claro"
sarcasmo = re.search(patron, texto.lower())

print("Sarcasmo detectado" if sarcasmo else "No detectado")
```

✓ 0.0s

Sarcasmo detectado

```
import re

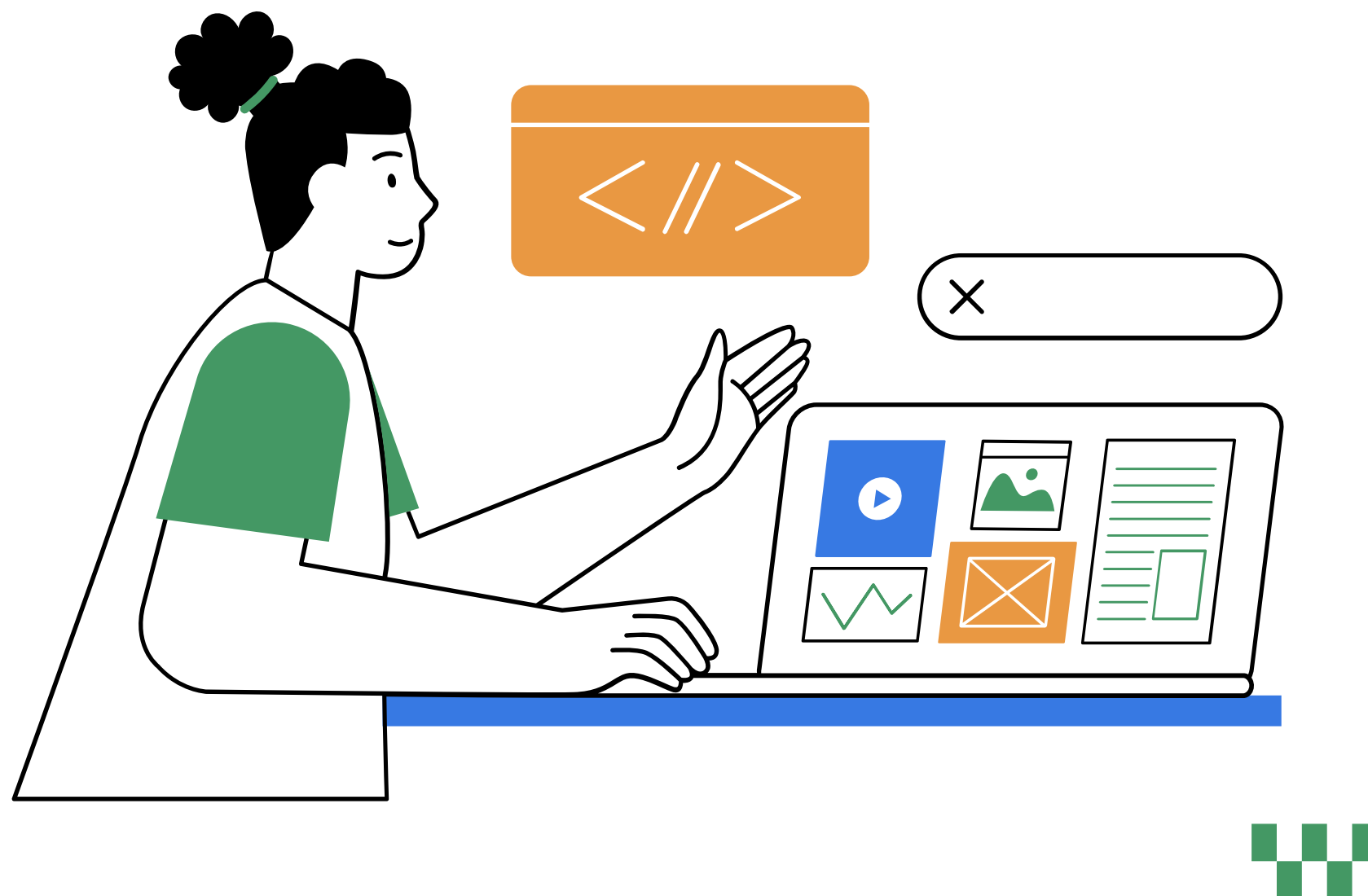
texto = "Obvio.... ya sabía eso."

patron = r"obvio\.{3,}"
sarcasmo = re.search(patron, texto.lower())

print("Posible ironía" if sarcasmo else "No detectado")
```

✓ 0.0s

Posible ironía

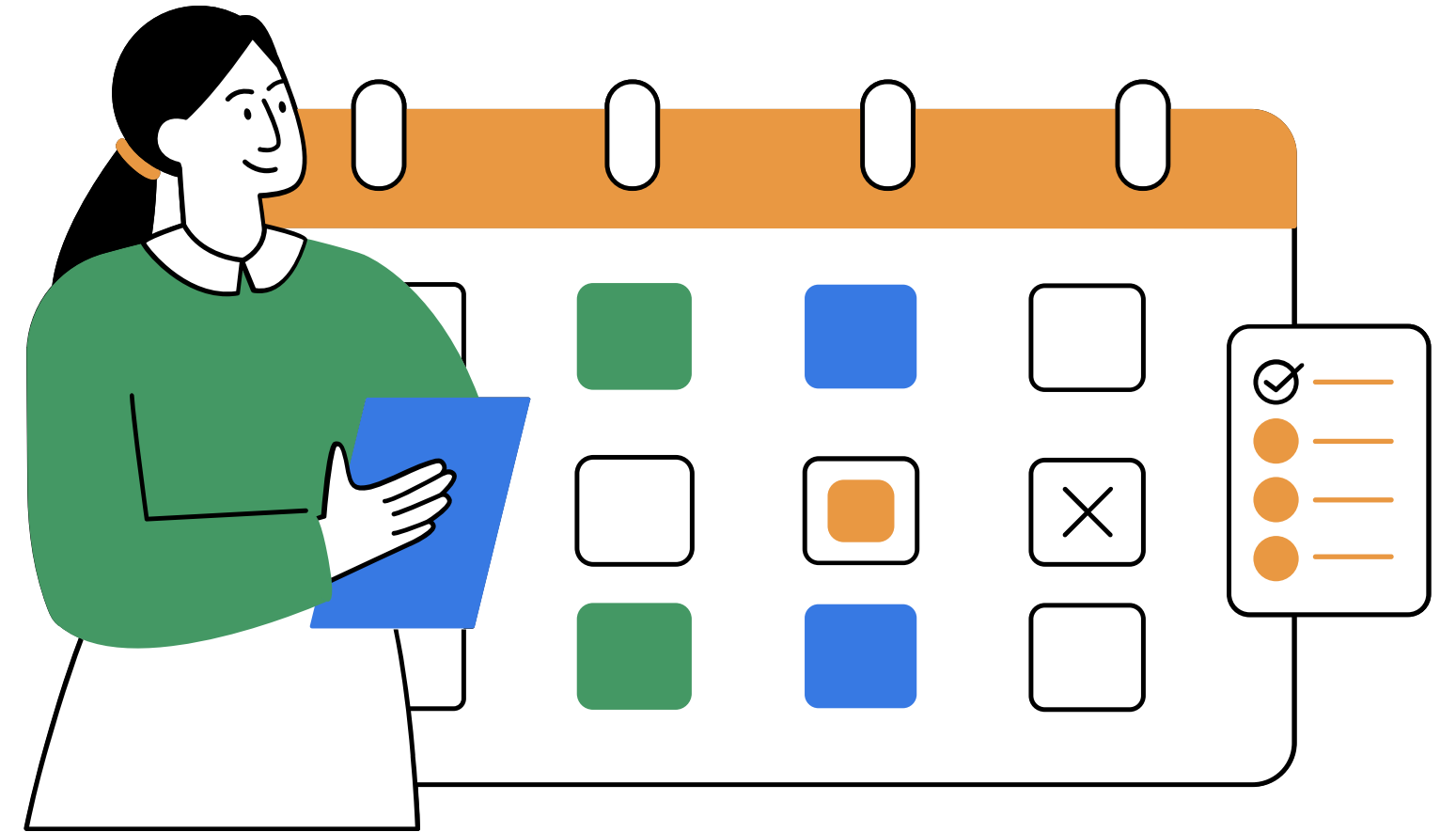


Text Summarization

Técnica mediante la cual podemos sintetizar fragmentos largos de texto en fragmentos de textos más cortos que contengan únicamente aquella información que es relevante

- **Técnica de NLP para reducir textos largos**
- **Mantiene la idea principal del contenido**
- **Elimina información redundante**
- **Facilita el análisis posterior**

Tipos de Text Summarization



Extractive Summarization

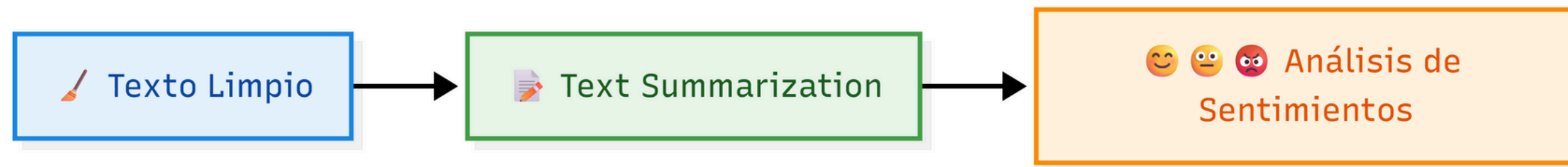
- Selecciona frases originales del texto

Abstractive Summarization

- Genera nuevas frases
- Similar a cómo resume un humano

Rol dentro del Análisis de Sentimientos

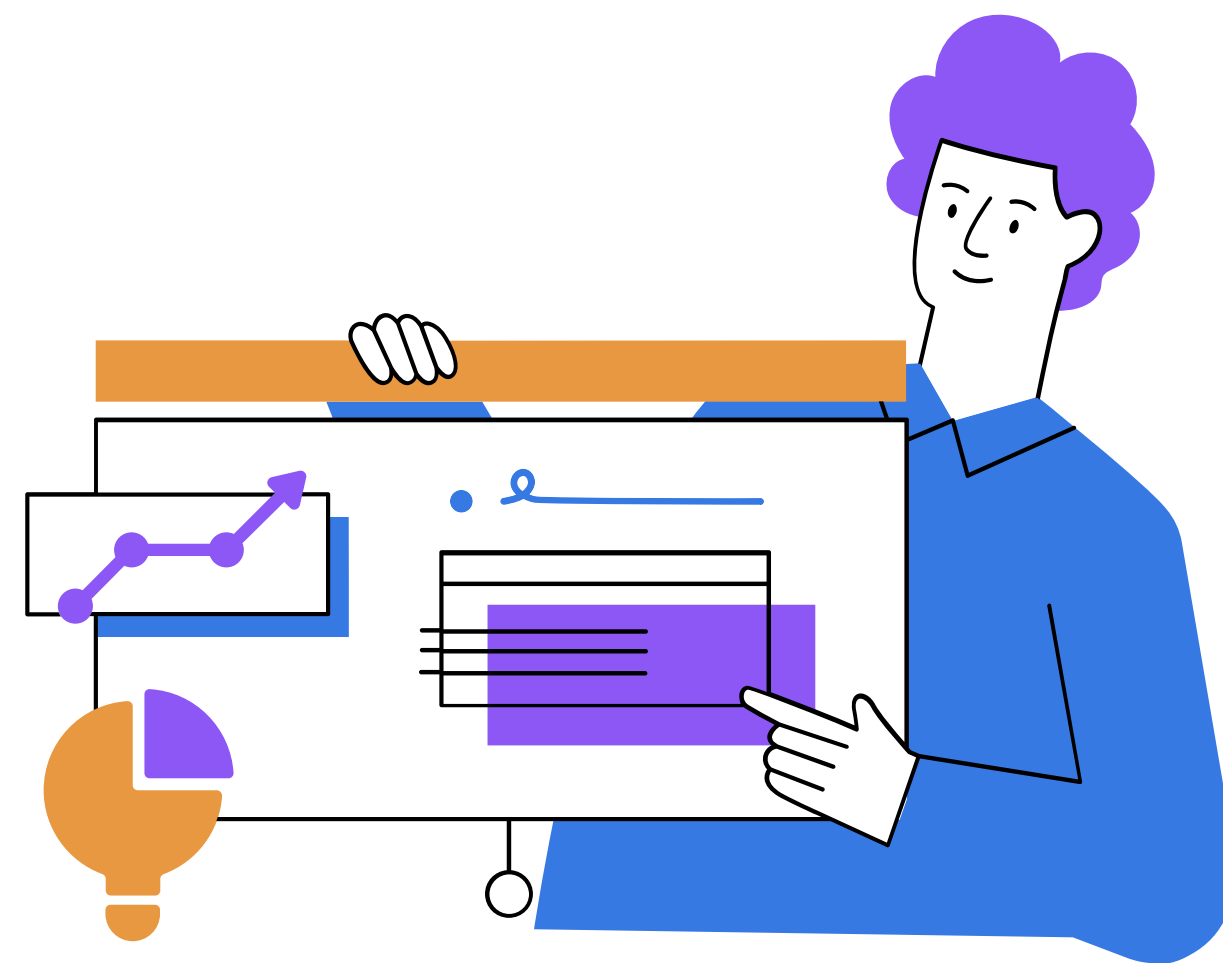
- Reduce longitud del texto
- Disminuye ruido
- Mantiene el sentimiento general
- Optimiza el análisis con Transformers





Transformer Models

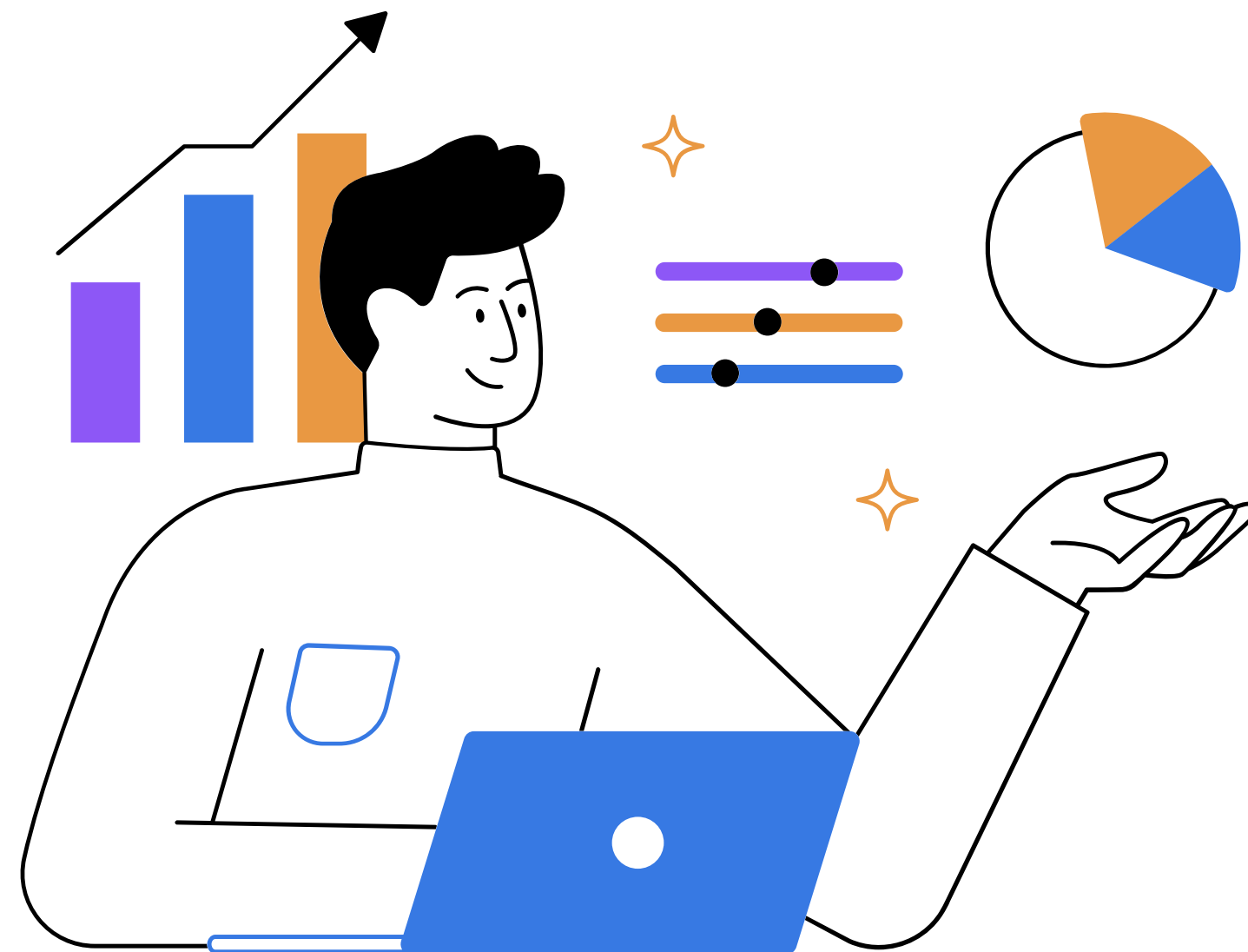
Son una arquitectura de red neuronal introducida por Google en 2017 la cual se caracteriza porque procesa datos en paralelo, no de forma secuencial (como hacían las antiguas Redes Neuronales Recurrentes). Son la base de todo lo que conocemos hoy como IA moderna (GPT, BERT). En minería de datos, permiten procesar volúmenes masivos de texto mucho más rápido que los métodos tradicionales.



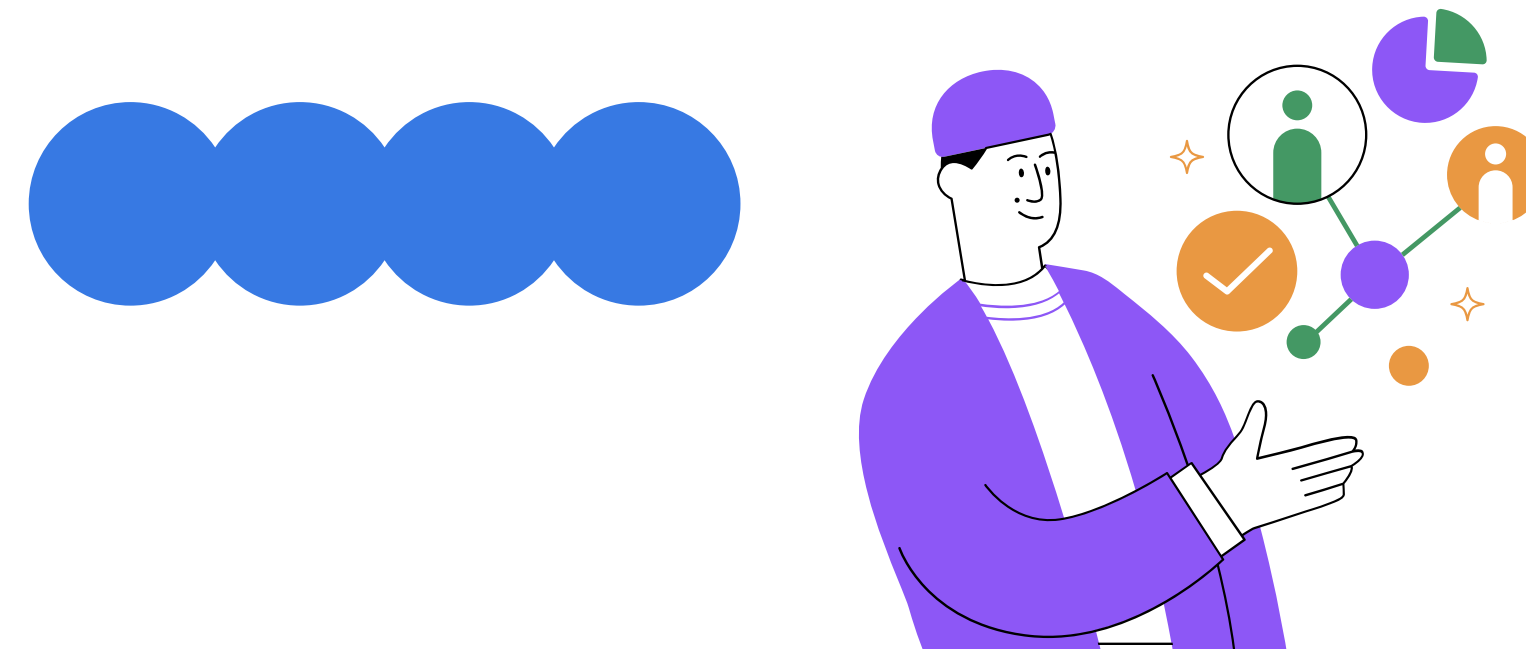
Este mecanismo funciona de manera que el modelo asigna un "peso" o importancia a cada palabra en una frase en relación con las demás. A diferencia de las expresiones regulares (que buscan palabras sueltas), el Transformer entiende que en la frase "La batería del celular es malísima", la palabra clave "malísima" va vinculada a "batería".

Esto nos permite capturar relaciones de largo alcance en un texto, algo que los modelos anteriores olvidaban.

Mecanismo de "Self-Attention"



Ventajas en Análisis de Sentimientos

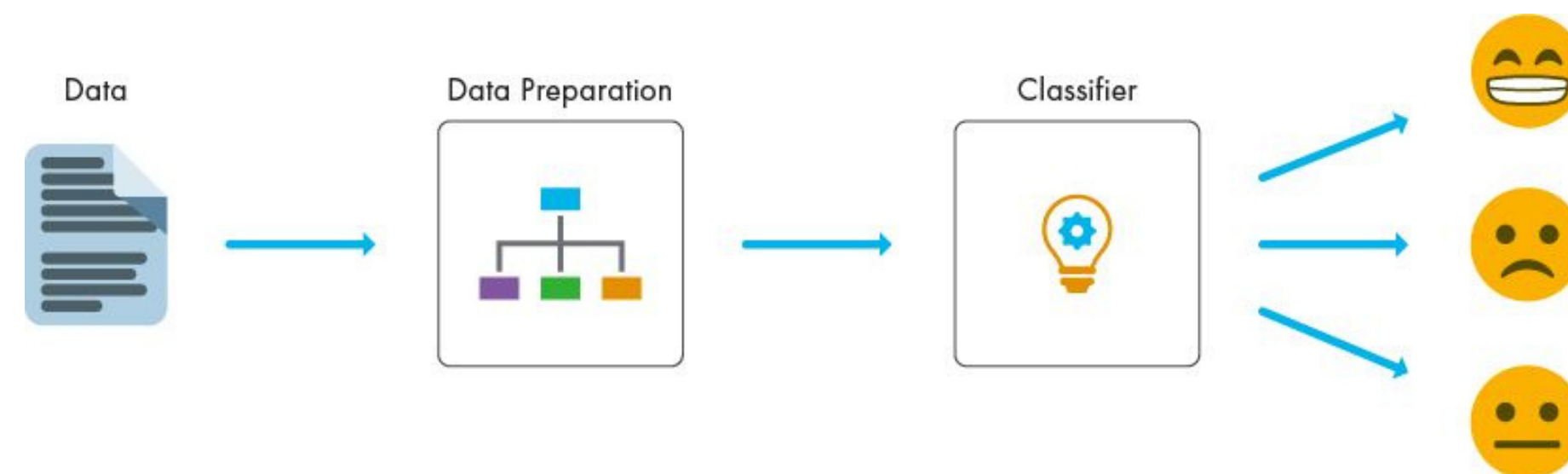


Bidireccionalidad: Los Transformers (como BERT) leen el texto en ambas direcciones al mismo tiempo. Esto ayuda a entender matices, sarcasmos o negaciones complejas.

Transfer Learning : No necesitas entrenar un modelo desde cero. Tomas un Transformer ya "entrenado" con todo internet y lo adaptas a tu conjunto de datos específico (por ejemplo, reseñas de Amazon o tweets).

Resultado: Mayor precisión en la clasificación de sentimientos (Positivo, Negativo, Neutral).

Antes de los Transformer Models, la minería de datos en texto era muy rígida y se perdía mucho contexto; con Transformers, la máquina "entiende" casi como un humano.



Thank You

